Donghyung Lee

DevOps Engineer | Career Description

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- AWS EKS, GCP GKE production operations & cost optimization (20% reduction via hybrid transition, 50% via GCP migration, savings reinvested in new projects)
- IaC-based infrastructure automation and version control using Terraform (100% GCP resource codification)
- On-premises to cloud hybrid architecture design and migration experience

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- CI/CD pipeline design and enhancement using GitHub Actions, GitLab CI, ArgoCD
- 50-67% deployment time reduction with 3x weekly deployments shortening Time to Market, 95% rollback time decrease (30min → 1-2min)
- Jenkins-based Unity Android/iOS build automation and Slack-integrated deployment notifications

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- Integrated metric/log monitoring with PLG Stack (Prometheus, Loki, Grafana)
- Evolved from ELK → EFK Stack, then redesigned into ECK Operator-based Elastic Stack 9.0 CRD declarative management (automatic TLS rotation, automated rolling upgrades)
- Reduced MTTR and improved service availability via SLI/SLO-based alerting
- kube-prometheus-stack with custom alert rules, Grafana dashboards, and 4 DB exporters (MySQL, Redis, Elasticsearch, PostgreSQL) integrated

Automation & Efficiency

Operations automation and developer productivity tools using Python and Bash

- 90%+ manual work reduction through data collection, deployment, and document management automation
- Internal LLM service (Ollama + Open WebUI) ensuring 100% internal security guideline compliance and dev team productivity improvement
- Internal tool development: GitLab Webhook doc sync, Slack Bot APK deployment notifications
- Established self-sufficient Kubernetes ops at DP (Delivery Partner) sites — designed and ran a 6-month training program (avg 10 attendees, 4h/session), standardized materials reused across other clients

Kubernetes Platform Modernization & Open-Source Contribution

Zero-downtime migration of network/logging stacks and releasing reusable public charts as open source

- Migrated multiple ingress-nginx instances to a single NGINX Gateway Fabric control-plane + per-class Gateway CRs with zero downtime (full cutover, wildcard TLS consolidated)
- Migrated the ECK Operator-based logging stack from a local chart to an OCI chart with zero downtime (CR/PVC names preserved, cluster_uuid unchanged, no data loss)
- Released 3 open-source Helm charts (nginx-gateway-cr, elasticsearch-eck, kibana-eck) and multiple composite GitHub Actions on ArtifactHub / GitHub Marketplace

Data Engineering & Internal LLM Services

End-to-end ownership from GCP data pipelines to in-house LLM services

- Built a multi-source (MongoDB · CloudSQL · GA · Dune) → BigQuery → Google Sheets pipeline by combining BigQuery, Dataflow, Cloud Functions, and Cloud Scheduler — entire infrastructure 100% Terraform IaC
- Deployed an internal LLM service (Ollama + Open WebUI) on a Kubernetes-hosted on-premises GPU server, saving ~$100/month by replacing external AI SaaS subscriptions and ensuring 100% internal security compliance
- 95% data-collection automation and 0% data gap rate by eliminating human error

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As the sole DevOps infrastructure engineer at a game development company, I design, build, and operate AWS-based cloud and on-premises servers. Development and test servers are on-premises, while QA and Production servers are on AWS, operating a hybrid architecture that reduced monthly infrastructure costs by 20% ($60K → $48K) and channeled the savings into new project investments.
Currently providing secondary technical support and operations for a live-service game in collaboration with the external publisher, and building on-premises/AWS infrastructure to improve development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution 100% (Solo design & implementation, in collaboration with the external publisher's ops team) 2024.10 ~ Present (In Operation)

Problem

While operating the entire infrastructure on AWS, high cloud costs of $60,000/month necessitated cost optimization.

Approach

Workload-specific cost analysis revealed that dev/test environments had minimal traffic variation, making cloud auto-scaling unnecessary, while only QA/Production required scalability and stability. Designed a hybrid architecture transitioning dev/test to on-premises while keeping QA/Production on AWS.
The two environments are operationally independent without VPN (application traffic isolated); the image registry is also split by environment (dev = Harbor, QA/Production = ECR), with each environment building independently to match its architecture. Unified operational standards through Terraform and Ansible IaC integration (100% IaC coverage for new components, IAM excluded) across both environments, achieving consistent infrastructure management without increasing operational complexity.

Troubleshooting

- When configuring ALB Ingress on EKS, needed to route to different backends per game client version.
  Resolved by combining custom header-based routing rules with Target Groups in ALB conditional routing
- Intermittent timeouts during ClusterIP service communication in Kubernetes IPVS mode.
  Resolved through Cilium eBPF packet flow analysis and IPVS conntrack threshold tuning, and built a Cilium Network Policy-based zero-trust security model to enhance inter-pod traffic visibility and security
- During rolling deploys, the ALB target deregistration delay (lowered from the default 300s to 60s by operational policy) didn't line up with Kubernetes Pod SIGTERM handling, causing some session connection drops. Solved via 2 changes — achieving zero-downtime deploys with 0 game-session connection drops:
  · (1) Added a 60s sleep in the Pod lifecycle.preStop hook so the readiness probe fails first — making ALB mark the target unhealthy and stop sending new requests
  · (2) Raised terminationGracePeriodSeconds to 90s (ALB drain 60s + in-flight handling 30s)

Results

- Workload analysis-based infra optimization reducing monthly costs by 20% ($60,000 → $48,000), reinvesting ~$144,000 annual savings into new project infrastructure for resource reinvestment cycle
- Offloaded dev servers to on-premises, cutting ~$1,000/month in AWS resource costs and optimizing resource usage
- Ensured operational stability and fault isolation through environment role separation

Tech Stack

AWS (EKS, EC2, ALB, Route53, ACM, EFS, Aurora MySQL, ElastiCache, CloudFront), Kubernetes, Terraform, Helm

Project 2: On-premises CI/CD Pipeline Construction & Enhancement

Contribution 100% (Full infra & pipeline ownership) 2024.12 ~ Present (In Operation)

Problem

Developers manually built and deployed to servers, taking an average of 30 minutes per deployment with frequent failures due to human errors.

Approach

Built a GitOps-based automated deployment environment combining GitLab CI and ArgoCD. Developers simply Git Push to trigger automatic build→test→deploy, with ArgoCD monitoring Kubernetes cluster state for declarative deployments.
Eliminated Docker-in-Docker dependency using Kaniko for Docker image builds, and configured multi-environment (alpha/review/dev/staging) deployments in a single pipeline.

Troubleshooting

- When `syncPolicy.automated.selfHeal=true` was active on an ArgoCD Application, the controller immediately reverted operators' temporary hot-fixes, making incident debugging difficult.
  Split the policy by environment — prod: `selfHeal=false` with manual sync windows; dev/staging: `selfHeal=true` — and integrated Slack notifications for every sync event to gain both change traceability and stability
- Package installation failure on CI Runner due to GPG key expiration after GitLab upgrade. Standardized the apt-key renewal procedure into a runbook for fast recovery on recurrence
- Build time spikes due to cache invalidation during Kaniko builds.
  Stabilized build times by combining `-cache=true --cache-ttl=24h --snapshot-mode=redo` options to improve cache hit rate

Results

- 67% deployment time reduction (30min → 10min)
- 90% manual work automation (build, deploy, config apply)
- Deployment automation increased weekly deployments 3x+ (1-2 → 5-7), shortening Time to Market for new features, while deployment resource efficiency enabled the dev team to focus on core feature development

Tech Stack

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

---

Project 3: Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement)

Contribution 100% (System design & implementation) 2024.11 ~ Present (ELK → EFK → ECK 3 Phase, ~18 months total)

Problem

Server and container logs were distributed across the on-premises environment, making root cause analysis time-consuming during incidents, with no unified monitoring system in place.

Approach

Phase 1 (ELK Setup): Built a centralized logging system using Elasticsearch, Logstash, Kibana, and Filebeat. Collected logs from all servers and containers via Filebeat, processed JSON parsing and field mapping in Logstash, stored in Elasticsearch, enabling real-time monitoring and search through Kibana dashboards.

Phase 2 (EFK Migration): Migrated to Fluent Bit + Fluentd based EFK Stack due to Logstash's high JVM memory usage (1GB+), Filebeat's Helm ecosystem mismatch, and log pipeline custom extensibility limitations. C-based lightweight collector Fluent Bit runs as DaemonSet on each node with automatic Kubernetes metadata tagging, while Fluentd handles log processing/filtering with multi-output support before forwarding to Elasticsearch. Developed a chart upgrade automation script to preserve custom PVC templates during upstream Helm chart auto-upgrades.

Phase 3 (ECK Operator Migration + Stack 9.0 Major Upgrade): With Elastic's official Helm Chart stalled for over two years and CRD-based declarative management (ECK) becoming mainstream, performed a major upgrade of Elastic Stack from 8.5.1 to 9.0.0 and introduced ECK Operator 3.3.2 in the elastic-system namespace.
Redefined Elasticsearch/Kibana as Custom Resources managed via a local chart (CR wrapper) to align with repo convention, and adopted a parallel deployment (monitoring → logging ns) and cutover strategy for zero-downtime migration.
Under the ECK-based model, established automatic TLS certificate issuance/rotation (1y / 30d), elastic account password management via Helm templates, and a rolling upgrade model completed by a single-line CR spec.version change — eliminating the manual operational burden.

Troubleshooting

- Logstash parsing failures due to null bytes (\x00) in game-server logs. Resolved by adding a preprocessing step using gsub in mutate filter
- Field mapping conflicts (mapper_parsing_exception) from different data types reaching the same field name.
  Resolved with explicit index-template mappings + Logstash type conversion filters
- Indices flipped to read-only when log volume spikes pushed Elasticsearch past its disk watermark. Stabilized with an ILM policy that auto-deletes indices older than 7 days
- [EFK] Fluent Bit failed to write WAL files on NFS volumes.
  Resolved by injecting a custom PVC volume patch into the upstream Helm chart's Pod template and auto-preserving it through the upgrade script
- [EFK] Log loss due to Fluentd output buffer overflow. Resolved by tuning chunk_limit_size + flush_interval and setting overflow_action=block
- [ECK] Kibana 9 enforces the Secure flag on session cookies when `server.ssl.enabled=true`, blocking login on HTTP.
  Restored HTTP login with `tls.disabled + publicBaseUrl http:// + secureCookies:false`
- [ECK] Fluentd's official image had no ES9 variant, forcing the ES8 variant.
  Achieved ES 9 compatibility via `fluent-plugin-elasticsearch 5.4.4` + `suppress_type_name:true`, then verified the fluent-bit → fluentd → ES path end-to-end by injecting an active marker log into the NFS PVC
- [ECK] 3.3.x stackconfigpolicy-controller repeatedly GET-ed resources outside managedNamespaces every 17 minutes, emitting reconcile errors.
  Worked around by setting managedNamespaces:[] (cluster-wide watch); RBAC was already cluster-scoped so no permission delta

Results

- 90% log search time reduction (per-Pod access → single Kibana interface), improved MTTR keeping service availability stable
- EFK migration removed Logstash's JVM memory burden (1GB+) by redesigning into a Fluent Bit DaemonSet + Fluentd architecture; now processes 1GB/day in real time with 90-day retention
- ECK Operator migration removed manual operations for TLS certificates, account passwords, and StatefulSet upgrades — rolling upgrades land via a single-line CR spec.version change
- Completed an Elastic Stack 8.5.1 → 9.0.0 major upgrade, closing a 2-year update gap
- ECK Operator's secureMode + ServiceMonitor auto-registers controller-runtime metrics into kube-prometheus-stack, extending observability into the Operator itself

Tech Stack

Elasticsearch, Fluent Bit, Fluentd, Logstash, Kibana, Filebeat, ECK Operator, CRD, Custom Resource, Kubernetes, Helm

---

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server)

Contribution 100% (Full system design & development) 2025.02 ~ Present

Designed, developed, and operated 5 internal tools to automate repetitive manual tasks and boost development team productivity.

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Problem

Manual APK delivery to QA team after build completion caused sharing delays and version confusion

Solution

Developed Python bot that auto-sends build completion messages with download QR codes to Slack when Jenkins APK builds finish, deployed on Kubernetes

Results

90% QA team delivery time reduction, eliminated version confusion with QR code-based instant install, automated build history logging

Tech Stack

Python, Slack Bot API, Jenkins, Kubernetes, Docker

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.06)

Problem

Dual work of writing technical docs in GitLab markdown then manually uploading to Google Drive

Solution

Integrated GitLab Webhook with Google Drive API for auto-sync on push. Python converts markdown → Google Docs for upload

Results

80% document management time reduction (10min → 2min per doc), unified GitLab as Single Source of Truth

Tech Stack

GitLab Webhook, Google Drive API, Python, Bash Script

4-3. Internal LLM Service Deployment (4 weeks, 2025.11)

Problem

Growing LLM demand from dev team, cost burden and internal data security concerns with external SaaS

Solution

Deployed Ollama and Open WebUI on Kubernetes on on-premises GPU server. Configured model storage with NFS

Results

Serving 15 developers with an average of 100 queries/day for code review, document summarization, and debugging assistance. Authored a user guide and personally onboarded the 15-developer team to drive adoption.
Saved ~$100/month by replacing external AI SaaS subscriptions. 100% compliance with internal security guidelines and eliminated external data leakage risks

Tech Stack

Ollama, Open WebUI, Kubernetes, NFS, GPU Server

4-4. Git Repository Mirroring Tool + Retry API Follow-up (4-week initial build, 2026.02 ~ 2026.06)

Problem

Manual source-code synchronization across AWS CodeCommit, GitLab, and GitHub caused omissions and delays. Later, when mirrors failed in operation, the only manual-retry path required operators to `kubectl exec` into the container and reuse the webhook secret — exposing the secret and offering no automation surface.
The AWS eu-central-1 transient blip incident pushed a faster recovery tool to the top of the priority list.

Solution

Phase 1 (initial build, 2026.02): built a Go-based bidirectional Git mirroring tool git-bridge supporting SQS polling (CodeCommit) + HTTP Webhook (GitLab / GitHub) event sources, with incremental sync (git fetch) and loop detection. Backed by unit tests + a GitHub Actions CI/CD pipeline, deployed on Kubernetes with Slack alerting.

Phase 2 (Retry API follow-up, 2026.05): added a `POST /retry/mirror` endpoint with Bearer-token auth (`RETRY_API_TOKEN` , separated from the webhook secret, constant-time compared). The JSON body accepts repo / direction / ref; the API responds immediately and a background goroutine re-runs the mirror, with EventMeta.Source set to `retry-api` so Slack alerts are distinguished from webhook-triggered ones. Direction supports `auto / source-to-target / target-to-source` , with per-repo `retry_direction` overrides and per-repo Slack webhook env vars to split notifications across channels. Added 11 new tests.

Phase 3 (stability guard, 2026.06): root-caused a non-fast-forward regression where multiple uncoordinated force-pushes in a bidirectional mirror rolled a shared branch back to a stale snapshot, and designed a `ref_overrides` feature that pins the sync direction per ref pattern. The repo stays bidirectional while specific branches are pinned one-way, preventing reverse-direction echoes from rolling a branch back. Scoped pushes down to the triggered ref (removed `--all` , applied only to override repos so existing repos behave exactly as before) and applied the same direction restriction to delete events. Designed on a **fail-open principle** — the guard falls back to the existing (force) behavior on failure so a healthy mirror is never stalled.

Results

- Processes an average of 30 sync events/day across 10 repositories, fully automating multi-platform source sync and eliminating manual mirroring; open-sourced and utilized alongside GitHub Actions (multi-git-mirror)
- The retry API delivered a safe manual-retry path that no longer exposes the webhook secret; dev verified that `auto` direction resolves to `target-to-source` and the Slack TEST channel receives the message
- Passed the CI test gate and provided an SRE-friendly recovery handle for incidents like the AWS regional transient blip
- Implemented and verified a direction-pinning guard that blocks non-fast-forward regressions.
  Passed 6/6 runtime checks (regression · new-feature · comprehensive) on a test-only repo while keeping the production repo isolated and unaffected; landed 4 commits (feat / fix / docs / chore) on main

Tech Stack

Go, AWS SQS, GitLab Webhook, GitHub Webhook, HTTP Bearer Auth, Slack Webhook, Kubernetes, Docker

---

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

Problem

The previously used halverneus/static-file-server:v1.8.11 provided only basic file serving, lacking operational features such as directory listing UI, file preview, search/filter, and batch download.
Demands for improved UX in QA APK distribution and internal file sharing persisted, while the upstream chart maintenance had stalled.

Solution

Designed and developed a lightweight Go-based static file server (static-file-server) in-house and operated a Helm repository on GitHub Pages to establish a deployment standard.
Deployed to Kubernetes with an in-house Helm chart, connected to NFS storage for stable long-term artifact hosting, fully replacing the legacy halverneus/static-file-server deployment.

Key Features

- Dark-mode directory listing UI (grid/list view switching, keyboard navigation)
- File preview (image / video / audio / PDF / text · code)
- Search / filter + URL hash sharing, multi-select + ZIP batch download
- Gzip compression, Prometheus metrics, structured JSON logging, health check
- Automatic APK file Content-Type handling (via ingress annotation)

Results

Released as open source under Apache-2.0 (operating Docker Hub and GitHub Pages Helm repo). Handles an average of 30 downloads/day and 5 APK distribution builds/week (daily build), unifying various internal file-sharing needs — APK distribution, document sharing, build artifact delivery — into a single tool, and standardizing deployment / upgrade / rollback via the in-house Helm chart.
Eliminating the upstream dependency enabled managing security patches and new features on an in-house cycle.

Tech Stack

Go, Helm Chart, GitHub Pages, Docker Hub, Kubernetes, NFS, Prometheus

---

Project 5: Kubernetes Cluster Operations Enhancement

Contribution 100% (Solo design — monitoring · upgrade automation · Helm management framework · NGF migration · 3 OSS Helm charts · storage performance optimization · Shell→Python migration; captured as 4 automation scripts establishing a standard procedure → so the team can reuse)
2025.12 ~ Present

To advance cluster visibility, operational stability, network modernization, storage performance, and automation-code quality, overhauled monitoring, automated K8s upgrades and Helm management, and migrated the network controller onto the Gateway API.
Released the artifacts from the NGF migration and ECK Operator adoption as OSS Helm charts, resolved storage bottlenecks via control-plane / DB-node SSD migration and build I/O isolation, and modernized the infra deploy/upgrade pipeline with wave-based automation + a Shell → Python migration.

5-1. Monitoring & Alerting Enhancement (2025.12 ~ Present)

Problem

The default kube-prometheus-stack install lacked custom alerts and dashboards across the infrastructure/DB/network layers, and was missing metric collection for physical servers, VMs, and the Cilium CNI — making proactive failure detection difficult.

Solution

Designed custom alert rules and Grafana dashboards based on kube-prometheus-stack. Integrated MySQL/Redis/Elasticsearch/PostgreSQL exporters and deployed node-exporter on physical servers and VMs via Ansible.
Collected Cilium CNI agent/operator metrics via kubernetes_sd_configs and configured severity-based Slack routing with inhibit rules in Alertmanager to improve alert quality.
(Chose kube-prometheus-stack over SaaS Observability — Datadog / New Relic — because on-premises metric sovereignty, exporter freedom, and zero runtime cost mattered more than turn-key dashboards.)

Results

Built proactive failure detection across infrastructure/DB/network layers with custom alert rules and dashboards, and extended unified observability coverage to physical servers and VMs.
Severity-based routing and inhibit rules removed duplicate alerts, improving on-call response quality.

Tech Stack

Prometheus, Grafana, Alertmanager, Cilium, node-exporter, Ansible, Slack API

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ Present)

Problem

Kubespray-based K8s upgrades were performed manually, making pre-flight validation, backup, and compatibility checks inconsistent, and with no versioning standard across the infrastructure Helm charts, upgrade scripts drifted per chart.
Kubernetes certificates expired annually, exposing the API Server to outage risk.

Solution

Built a Kubespray version management automation script (pre-flight validation, inventory backup, version compatibility check, breaking change detection) and a multi-step post-upgrade health check script (node/Pod/DNS/cert/etcd/API Server).
Built an upgrade script framework across all infrastructure Helm charts along with a synchronizer that manages four canonical templates as a single source of truth (check mode detects drift in CI, apply mode propagates in bulk).
The automated Kubernetes certificate renewal script supports a stepwise flow (verify → backup → renew → kubelet/apiserver restart → kubeconfig update → final verification) with rollback, scheduled via crontab across the nodes every 6 months at 3 AM.
(Kept Kubespray because kOps is AWS-only and Cluster API's on-premises bootstrap is heavyweight — sticking with Kubespray let the existing Ansible-based ops absorb only the new automation layer.)

Results

The multi-step automated health check reduced post-upgrade verification time by 90%, while the upgrade framework + canonical template synchronizer across the Helm charts standardized infrastructure version management and enabled CI-based drift detection of script bodies.
The automated certificate renewal script with crontab scheduling eliminated API Server outage risk from certificate expiration across the nodes.

Tech Stack

Kubespray, Ansible, Helm, etcd, Shell Script, GitLab CI

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Problem

On the on-premises cluster, Multiple ingress-nginx instances (default + public-a~j, each with a pinned MetalLB LB IP) duplicated Deployment / Service / CRD / RBAC per class for ingressClassName-based multi-tenancy.
With the Kubernetes Gateway API reaching v1.2 GA and ingress-nginx slated for maintenance mode, the controller had to be migrated to the Gateway API-based successor.

Solution

Adopted the official successor NGINX Gateway Fabric (NGF) 2.x to collapse the topology into a single control-plane + per-class Gateway CRs. Preserved MetalLB IPs so DNS/firewall stayed unchanged via parallel deployment + gradual cutover, and completed the full Phase 0~7+ pipeline (MetalLB pool expansion → NGF install → observability stack → HTTP-only / ApplicationSet apps → HTTPS-enforced apps → IP-swap cutover → Ingress cleanup) **within two days**.
Mapped ingress-nginx annotations 1:1 to Gateway API + NGF CRDs (HTTPRoute filters / ClientSettings·ProxySettings·RateLimit·BackendTLSPolicy) and unified the HTTPRoute values schema across the charts so ApplicationSet apps could migrate in batch. Consolidated per-app self-signed TLS Secrets into a single wildcard cert (internal domain, 10-year validity).

Troubleshooting

- With `externalTrafficPolicy: Cluster` + a pinned `loadBalancerIP`, MetalLB held the IP even after scaling Pods to 0, blocking the swap to NGF.
  Patched the Service to ClusterIP and dropped `loadBalancerIP` as a new step in the cutover script to force release
- NGF's chart default `externalTrafficPolicy: Local` caused 2-minute timeouts on internal Pod → public domain → LB IP paths (single dataplane replica → most nodes missed locally).
  Explicitly set `externalTrafficPolicy: Cluster` across the `NginxProxy` CRs to match the prior ingress-nginx policy, fixing CI generator regressions

Results

Collapsed multiple controllers into a single NGF control-plane + per-class Gateway CRs — eliminating CRD/RBAC/Deployment duplication — and completed all class cutovers with zero incidents (30-60s actual downtime per class).
Unified HTTPRoute values schema standardized the ApplicationSet batch migration, wildcard cert consolidation cut manual renewal overhead by 50%, and finishing Phase 0~7+ in two days minimized the migration window.

Tech Stack

NGINX Gateway Fabric, Gateway API, HTTPRoute, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy, Helmfile, MetalLB, ArgoCD

5-4. Three Open-Source Helm Charts — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

Problem

The local CR charts built during the NGF migration and ECK Operator adoption were tightly coupled to internal storageClass / CR / Secret names, blocking external reuse. The NGF upstream chart installs only the controller and CRDs and does not provide the tenant-level resources needed for actual traffic paths (Gateway, NginxProxy, ReferenceGrant, ServiceMonitor, PodMonitor), and Elastic's official Helm Chart had been stalled for 2+ years.
We needed to refine the internal artifacts into reusable public charts while also providing a no-data-loss migration path for clusters already running the local charts.

Solution

nginx-gateway-cr: refined the local CR chart extracted from the NGF migration (5-3) into a reusable chart that deploys five tenant-level resources — Gateway / NginxProxy / ReferenceGrant / ServiceMonitor / PodMonitor. Includes a multi-Gateway-friendly schema (`gateways[]` array with per-gateway overrides), PodMonitor templates recommended for NGF 2.x (controller + dataplane), and `values.schema.json` input validation.

elasticsearch-eck / kibana-eck: elasticsearch-eck (11 ES templates) + kibana-eck (10 Kibana templates) deploy the CRs, Secrets, HTTPRoute, Ingress, BackendTLSPolicy, ServiceMonitor, and NetworkPolicy from a single set of values. Added `values.schema.json` (draft-07) validation for HA / storage / resource inputs and documented Minimal / HA presets and per-environment storageClass mapping in the README.

Unified distribution: all three charts ship simultaneously across OCI (ghcr.io/somaz94/charts) + a gh-pages Helm repo + ArtifactHub (networking / monitoring-logging categories, Apache-2.0). Added a new canonical version-tracking template to tighten internal OCI chart-version tracking.

Zero-downtime cutover: when migrating the mgmt cluster from the local chart to the OCI chart (elasticsearch-eck / kibana-eck 0.1.1), every CR / PVC / Secret name was preserved — cluster_uuid stayed unchanged and the migration completed with zero Elasticsearch pod restarts and a single Kibana rolling restart, i.e. no data loss.

Results

- **Three charts** (nginx-gateway-cr, elasticsearch-eck, kibana-eck) **published on ArtifactHub under Apache-2.0** — turning internal NGF + ECK artifacts into reusable community assets
- Extended a **dual-use model — internal helmfile pipeline + external OSS distribution** — across both the networking and the logging stack
- HA rolling scenarios passed 3/3 on a local kind cluster; end-to-end install on the real cluster reached Ready/green in 71s for Elasticsearch and 57s for Kibana
- The new canonical OCI chart-version tracking template makes the same pattern reusable for any future OCI chart

Tech Stack

Helm Chart, Gateway API, NGINX Gateway Fabric, ECK Operator, Elasticsearch, Kibana, Prometheus Operator, OCI Registry, ArtifactHub, Kubernetes

---

5-5. Storage Performance Optimization — etcd / DB SSD Migration + Build I/O Isolation (2 weeks, 2026.04 ~ 2026.05)

Problem

On the on-premises cluster, the control-plane VM and the game-DB-hosting worker VM were both backed by HDDs, contending with GitLab Runner buildx jobs for disk I/O. etcd WAL fsync p99 spiked to ~2s, apiserver pods GET p99 to ~7s, and the game DB COMMIT latency exploded to 9~14s — causing recurring play-blocking incidents.
Root cause analysis pinned it on GitLab Runner's heavy write traffic saturating the game-DB worker VM's disk fsync queue.

Approach

Step 1 (immediate mitigation): tuned MySQL with 6 parameters (`innodb_flush_log_at_trx_commit=2`, `innodb_flush_method=O_DIRECT_NO_FSYNC`, ⋯) to temporarily ease fsync pressure.

Step 2 (control-plane VM SSD migration): sparse-copied the KVM/libvirt VM's qcow2 (11GB, 6m50s) onto SSD, then completed the cutover via virsh XML edit + cold start with 8.5 minutes of downtime (vs. a 15-minute estimate). Verified via etcd health checks and Prometheus metrics right after start-up.

Step 3 (game-DB worker VM DB/Redis SSD migration): lossless migration of 35GB across game-db (MySQL) and dev/qa/stg valkey-redis via stop → backup → KVM SSD migration → restore → local-path PV rebind.

Step 4 (build-only VM): provisioned a new VM on a separate physical server — 4 vCPU / 16GB RAM / 150GB SSD — via cloud-init, joined it to the cluster via kubespray, and applied `node-role=build` label + `dedicated=build:NoSchedule` taint. Added nodeAffinity + tolerations to the GitLab Runner Helm values so buildx jobs land on the build-only node.
Packaged the work into 4 reusable scripts plus config files so future worker nodes follow the same procedure.

Troubleshooting

- cloud-init VMs failed to bring up the network interface because YAML was parsing the MAC `0e:bd:01:51:00:0a` as sexagesimal and dropping the trailing `0a`.
  Quoting the MAC field in cloud-init network-config and adding a MAC-match validation step to the VM verification script prevented recurrence
- Right after the control-plane VM cold start, etcd briefly re-entered leader election and apiserver returned 5xx for ~30s.
  Eased the kube-apiserver readiness threshold beforehand and added an explicit 'etcd quorum recovery check' gate into the migration runbook

Postmortem & Recurrence Prevention

- Promoted the MAC sexagesimal pitfall fix into a runbook gate — quoted MAC fields in cloud-init network-config + a MAC-match check in the VM verification script now block any VM-build regression
- Promoted the etcd leader-election 5xx finding into an explicit 'etcd quorum recovery check' gate in the migration runbook, blocking the same 5xx exposure on future VM SSD migrations
- Packaged the 4 scripts + config files as a standard procedure asset — next worker-node addition reuses the same validation gate (11 PASS)

Results

- **etcd WAL fsync 1760ms → 3.88ms (454× faster)**, **etcd backend commit 1810ms → 3.77ms (480× faster)**, **apiserver pods GET p99 6520ms → 23ms (283× faster)** — Kubernetes API availability restored
- **Game DB COMMIT latency dropped from 9~14s back to the ms range**, taking play-blocking incidents during GitLab Runner builds down to zero
- build-only VM validated with 11 PASS / 0 WARN / 0 FAIL, achieving physical isolation between build and DB workloads
- Captured the work as 4 automation scripts + config files so future worker-node additions reuse the same standard procedure (ops standardization)

Tech Stack

KVM / libvirt, qcow2, virsh, etcd, Prometheus / PromQL, MySQL, Redis (valkey), kubespray, cloud-init, GitLab Runner

5-6. Infra Deploy/Upgrade Automation + Shell → Python Migration (2026.03 ~ Present)

Problem

The 25 components in the internal Kubernetes infra monorepo were managed by ~39,000 lines of helmfile + shell, leaving new-component deploys, chart upgrades, and canonical-template sync entirely manual.
Shell + awk hit walls on multi-environment branching, complex YAML patching, and testability, and the CI environment took ~5 minutes just to install helmfile and its plugins — a real per-job cost at scale.

Approach

Deploy automation: structured the CI pipeline into 6 deployment waves (bootstrap → core → security → db-redis → observability → apps) — MR diff detection → helmfile diff/apply only on changed components, with a manual gate making the prod-safe wave-based deploy a standard. Reorganized 10 CI scripts (helmfile-changed-dirs, diff-component, apply-component, auto-upgrade, render-mr-body, ...).

Upgrade automation: CI detects new versions across all 25 component charts and opens an MR with auto-rendered body — changed chart, values diff, and breaking-change notes.

Shell → Python migration: modularized 8 canonical templates (external-standard / external-oci / local-with-templates / cr-version, ...) using stdlib only (pathlib, subprocess, re, json, argparse) — also rewrote the sync and backup scripts + 6 CI scripts (1,387 lines total) in Python. Built up a per-component upgrade module (25 total) and 654 unittests alongside.

helmfile-tools image adoption: applied the layered toolchain image from Project 7 to bring CI install time from 5min down to ~10s. The same code runs both locally (macOS venv) and inside the CI container.

Results

- **Wave-based automated deploy of 25 components + auto-upgrade MR generation**, slashing the operational burden of infra deploys and upgrades
- **Migrated shell → Python** (~39,000 lines of helmfile + shell in the internal Kubernetes infra monorepo, ~50,000 lines including external automation shell; 654 unittests passing), gaining both readability and testability across multi-env branching and complex YAML patching
- Adopting the helmfile-tools image brought **CI bootstrap time from 5min to ~10s (~95% reduction)**, eliminating per-job cumulative cost
- Local (macOS venv) and remote (CI container) execution share the exact same code, removing environment-drift issues between dev and deploy

Tech Stack

Python 3.10+ (stdlib only), unittest, GitLab CI/CD, Helm / Helmfile, Git automation, yq

---

Project 6: Infrastructure Security System

Contribution 100% (Design, deployment, SSO integration) 2026.04 ~ Present

Building security infrastructure for safe management of sensitive information required for infrastructure operations.

6-1. Vaultwarden Password Management System (1 week, 2026.04 ~ Present)

Problem

Internal secrets and credentials were scattered across Slack / email / personal vaults, accumulating exposure and duplicate-management risks. External SaaS password managers would add ongoing cost and externally store internal data.

Solution

Deployed Vaultwarden (Bitwarden-compatible server) on Kubernetes via Helm with GitLab SSO (OpenID Connect) integration so existing accounts kept logging in. Managed self-signed TLS certificates through Helm extraObjects and applied organization/collection-based team RBAC.
Built an automated backup CronJob (daily SQLite dumps, 30-day retention) and an interactive restore script for data reliability.

Results

- Migrated 70 users and 50 secrets into centralized management, eliminating scattered-storage risk
- Kept external SaaS subscription cost at zero, avoided externally storing internal data
- Automated backup/restore procedure minimizes operational burden on internal secret assets

---

6-2. Keycloak Operator-based Central SSO Introduction (2026.04 ~ Present)

Problem

Internal infra tools — ArgoCD, Harbor, Vaultwarden — were each using GitLab directly as their OIDC IdP, layering more and more dependence on GitLab's own user / session / MFA policy.
To prepare for central management of LDAP / MFA / session policies, a dedicated central auth broker (Keycloak) had become necessary.

Approach

Introduced the Keycloak Operator + KeycloakCR + KeycloakRealmImport with a PostgreSQL backend, then configured Keycloak to broker GitLab as an Identity Provider so existing accounts kept logging in. Codified Realm / Client / IdP / Group / Mapper setup via a `kcadm.sh` bootstrap script and migrated Harbor and ArgoCD's OIDC config from direct-GitLab to via-Keycloak (Vaultwarden held off).
For RBAC, GitLab groups (`server`, `global-admin`) are brokered through Keycloak group mappers and wired straight into ArgoCD's role mappings, consolidating the source of truth for permissions onto Keycloak.

Troubleshooting

- Initially mis-set the IdP `providerId` to `oidc`, breaking GitLab brokering — corrected to `providerId: gitlab` (custom provider)
- GitLab IdP's `defaultScope` only contained `openid`, dropping the group claim — added `read_user` plus group-related scopes so ArgoCD RBAC works correctly
- Mismatch between the mapper's `claim.value` and the KeycloakRealmImport YAML schema kept the mapper config from applying — adjusted the key shape under `config:` to match the Keycloak Operator spec
- The Operator wasn't idempotent on a few Realm fields (e.g. client.secret), causing infinite re-creation on reconcile — split those out into externally managed Secrets so reconcile stays clean
- During Harbor's OIDC cutover, existing user mappings risked permission loss — pre-exported and verified Harbor's `oidc_user_id` mappings beforehand, achieving zero downtime / zero permission loss

Results

- **ArgoCD and Harbor OIDC integrations completed with zero downtime and verification checks PASS** — removed direct dependence on GitLab
- Established a single authorization flow from GitLab groups (`server`, `global-admin`) → Keycloak group mappers → ArgoCD roles, consolidating authorization SSOT onto Keycloak
- Captured five pitfalls hit during Keycloak Operator adoption (IdP providerId / scope / mapper config / Operator idempotency / Harbor user mapping) into the plan doc to prevent recurrence on future cluster rebuilds
- Laid the groundwork for future LDAP federation, MFA enforcement, and central session policy (additional Vaultwarden integration + LDAP federation pending follow-on triggers)

Tech Stack

Keycloak Operator, KeycloakCR / KeycloakRealmImport, PostgreSQL, OIDC / OAuth2, kcadm.sh, ArgoCD, Harbor, Kubernetes Gateway API

Results

- Migrated 70 users and 50 secrets into centralized management, eliminating security risks and improving handover efficiency
- Immediate login with existing company GitLab accounts via SSO integration, removing separate account provisioning/deprovisioning overhead and improving account management efficiency
- Introduced Keycloak central SSO — OIDC integration across 4 systems (Harbor, ArgoCD, etc.), removed direct GitLab-as-IdP dependency, 5 pitfalls captured as plan-document assets

Tech Stack

Vaultwarden, Helm, OpenID Connect, GitLab SSO, Kubernetes

Project 7: Shared Toolchain Image Layered Architecture + Auto Version Cycle

Contribution 100% (Architecture design · CI standardization · automation pipeline) 2025.09 ~ 2026.05

Problem

Across the 15 repos in the GitLab CI standardization scope (1 template provider + 14 consumers), each job re-installed helm / helmfile / kubectl / crane / aws-cli on top of an Alpine base, making first-job bootstrap take an average of ~5 minutes. Tool versions were scattered across the 14 consumer repos, so adding a new version effectively required a coordinated 14-repo bump.
Reliance on `:latest` tags also undermined reproducibility and made CVE tracking hard.

Approach

Layer 1 (Mirror): copied external registry images (Docker Hub, GHCR) into the internal Harbor via `crane copy`, preserving multi-arch manifests.

Layer 2 (Custom): built 7 fat images on top of Layer 1 (helmfile-tools, node-build, go-build, rclone-backup, aws-cli-tools, kaniko-build, terraform-tools) so consumer repos get the tools immediately on image pin alone.

Tier 1~2 SSOT: introduced a shared `.toolchain_build_custom` template + central `TOOLCHAIN_*_TAG` variables in gitlab-ci-templates, so consumer repos no longer track image tags directly.

Phase 4 auto cycle: a cron job drives the following 4 steps so the full cascade completes with just two manual clicks (pipeline trigger + Tier 2 MR merge):
· (1) Upstream version detection

· (2) Layer 1 mirror auto-MR
· (3) Layer 2 rebuild + Tier 2 TOOLCHAIN_*_TAG sync auto-MR
· (4) Auto-propagation to 14 consumer repos (excluding the template provider repo)

Results

- **First-job bootstrap time cut from 5min to ~10s (~95% reduction)**, with 100% tool-version consistency across 14 consumer repos thanks to the same shared standard (15 repos total including the template provider)
- Compressed 12+ scattered version points into a 4-tier governance flow (ARG → central var → consumer indirection → lint drift check)
- The Phase 4 cron auto cycle now lands new versions of helm / helmfile / kubectl / crane / rclone (5+ tools) with just 2 manual clicks, making version tracking effectively zero-overhead
- Adopted a minor-guard policy (helm/helmfile/kubectl) to prevent compatibility breakage in advance (e.g. validated helm >= 3.18.6 compatibility after helmfile 1.0.0 → 1.5.1 cascade)

Tech Stack

GitLab CI/CD, Docker Buildx (multi-arch), crane, Kaniko, Harbor, Helm / Helmfile, Bash / yq, Slack Webhook

---

Project 8: Fluent Bit Log Collector Data Integrity & HA Hardening

Contribution 100% (Tier 1~3 sole design & rollout — handed off to operators via 7 PrometheusRule annotations + Slack alert routing) 2026.03 ~ 2026.05

Problem

Operating the post-Phase-3 ECK stack revealed that whenever a Fluent Bit Pod restarted, its in-memory tail offsets were lost — risking either re-indexing the same logs or losing them entirely.
With only a single replica and no PDB / preStop hook, node drain caused brief collection gaps, and there were no alerts on the collector itself, leaving room for silent failure.

Approach

Tier 1 (data integrity): enabled a SQLite offset DB (`db /var/log/flb-storage/tail.db`) on the tail input and mounted a dedicated 2Gi PVC, so offsets survive Pod restarts. Verified on helmfile revision 4 — zero log re-index and zero duplicates.

Tier 2 (availability): added a PodDisruptionBudget (`minAvailable=1`), a preStop hook (`sleep + flush`), and liveness/readiness probes (/api/v1/health) to minimize collection gaps during node drain and rolling updates.

Tier 3 (monitoring): added 7 PrometheusRule alerts (input/output failure rates, retry buildup, fluentd queue pressure, fluent-bit Pod down, SQLite DB write failures, etc.) so silent failures surface proactively.

Tier 4 (HA option doc): documented three forward-looking HA options — input-path partitioning / sidecar collector / Vector multi-collector — with trade-off comparison.

Results

- Achieved zero log re-indexing and zero duplicates on Pod restart, guaranteeing log data integrity
- Minimized collection gaps during node drain / rolling update via PDB + preStop hook, contributing directly to MTTR
- Eliminated silent failures on the collector layer with 7 PrometheusRule alerts, establishing self-monitoring on the observability stack itself
- Documented three HA expansion options, reducing future decision cost as traffic grows

Tech Stack

Fluent Bit (tail / SQLite offset), Kubernetes PVC / StorageClass, PodDisruptionBudget, Prometheus Operator (PrometheusRule), Helmfile, Fluentd (buffer tuning)

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game/blockchain startup, leading the large-scale AWS → GCP cloud migration.
Drove the transition to capture GCP Startup Program cost optimization and GCP's global network performance, completing the full service migration within a scheduled game-maintenance window with minimal downtime. Maintained dual source management — GitLab for on-premises, GitHub for production.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Reconstruction

Contribution: Infra architecture design, migration strategy & execution lead 70% (remaining 30% = server-team application-side migration

collaboration), CI/CD pipeline construction 100% 10 months (2023.03 ~ 2023.12)

Problem

AWS costs were continuously rising ($10,000+/month), particularly NAT Gateway and data transfer costs.
A GCP Startup Program opportunity then prompted the migration — targeting cost reduction alongside the global load balancing needed for the multi-region game service.

Approach

Established a 3-phase migration strategy executed sequentially. Phase 1: Built and validated dev environment on GCP.
Phase 2: Migrated QA environment. Phase 3: Migrated production.
Designed the network on Shared VPC (Host Project / Service Project separation) and switched GitHub Actions' GCP auth to Workload Identity Federation, eliminating service-account key issuance/rotation overhead in favor of short-lived OIDC tokens. Migrated DNS via subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS) for pre-validation, then performed the final NS swap during a single scheduled maintenance window with minimal downtime. Worked around Filestore's 2.5TB SSD minimum cost by self-hosting an NFS server on Compute Engine (pd-balanced 1TB), minimizing operational cost. Configured Cloud Armor with region blocking + IP allow-list WAF policy and split the Cloud CDN URL Map into HTTP / HTTPS variants to enforce HTTP → HTTPS redirect alongside static-asset caching.
Codified GCP infrastructure with Terraform (100% excluding IAM), then rebuilt the GitOps CI/CD pipeline combining GitLab CI and ArgoCD. Standardized environment configs with Helm Charts and established Git commit-based deployment history management.

Troubleshooting

- While migrating AWS ALB-based routing to a GCP Global LB, traffic-routing issues arose because AWS ALB still passes traffic to backends failing health checks whereas GCP LB blocks them.
  Analyzed the GCP NEG (Network Endpoint Group) structure and reconfigured health checks/backends for GKE workloads
- GKE Ingress + BackendConfig health check returned 503 when the configured requestPath had no response.
  Resolved by guaranteeing the health-check endpoint response and aligning ManagedCertificate / FrontendConfig to restore external ingress traffic
- Game client file downloads failed after the Cloud CDN migration due to a missing HTTP → HTTPS redirect.
  Resolved by splitting the URL Map into HTTP / HTTPS variants (`default_url_redirect.https_redirect=true`) and mapping the HTTP forwarding rule to a dedicated port-80 target HTTP proxy
- After delegating DNS to GCP Cloud DNS, the domain was not automatically registered in Google Search Console, breaking search visibility and domain-ownership verification.
  Resolved by adding the Search Console verification TXT record to Cloud DNS to verify ownership, then re-registering the domain

Results

- 50% cloud cost reduction ($10,000 → $5,000/month), ~$60,000 annual savings
- Completed full service migration via the 3-phase strategy.
  Minimized downtime with resource-copy based migration, and executed the final DNS cutover during an approximately 2-hour scheduled maintenance window aligned with game service operations
- Established IaC management for entire GCP infrastructure with Terraform (ensured code quality with TFLint/Checkov static analysis and Terratest unit testing)
- 50% deployment time reduction (40min → 20min), 95% rollback time decrease (30min → 1-2min)
- 3x weekly deployments (2 → 6), 100% deployment history tracking via Git commits
- Eliminated GCP service-account key management overhead and key-leak risk by adopting Workload Identity Federation for GitHub Actions, transitioning to a short-lived token security model

Tech Stack

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB 등), Shared VPC, Workload Identity Federation, Cloud Armor, Cloud CDN, Cloud DNS, NFS, Terraform, GitLab CI, ArgoCD, GitHub Actions, Kubernetes, Helm, Docker

---

Project 2: Game Data Collection Automation System

Contribution 100% (Python script development & operations) 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, with analysts spending 2-3 hours daily on data collection and frequent data gaps due to human errors.

Approach

Built a GCP-based pipeline that loads multi-source data (MongoDB · CloudSQL · Google Analytics · Dune) into BigQuery and automatically refreshes analyst-facing Google Sheets. MongoDB was ingested via a Dataflow Flex Template ETL, CloudSQL via BigQuery's direct connection feature, and GA / Dune via their respective APIs (Dune with a separate API key), normalized into a consistent index schema.
Python Cloud Functions push BigQuery query results into Google Sheets via the Sheets API, and Cloud Scheduler triggers them on Daily (previous-day data) / Monthly (previous-month data) cadences. A separate dedup Cloud Function runs periodically to keep BigQuery data consistent. The full infrastructure (BigQuery datasets, Cloud Functions, schedulers, Cloud Storage, Artifact Registry, IAM bindings, service accounts) is managed as Terraform IaC for reproducible environments and change tracking.

Troubleshooting

- The pipeline depends on many GCP services (BigQuery, Dataflow, Cloud Functions, Cloud Scheduler, Sheets, Drive, etc.), so rebuilding in a new project failed when their service APIs were not enabled.
  Standardized enabling the required service APIs up front to prevent omissions on rebuild
- The CloudSQL → BigQuery direct connection (federated query) failed on region mismatch and missing connection-SA permissions.
  Resolved by creating the connection in the same region and granting the connection SA the Cloud SQL Client role
- Dune API's async execute → poll → fetch flow plus rate limits caused missing results / timeouts.
  Guaranteed result retrieval via execution-status polling with backoff and moved the API key into Secret Manager
- Daily re-runs re-loaded duplicate rows into BigQuery (non-idempotent).
  Applied MERGE / ROW_NUMBER() dedup logic in the Cloud Function to make loads idempotent, keeping zero gaps/duplicates
- Loading large BigQuery results into Google Sheets failed on the 10M-cell-per-sheet limit and write quota.
  Resolved with chunked values.batchUpdate writes, exponential backoff, and sheet splitting
- Continuously updating many sheets hit the Google Sheets (Drive-family) API per-minute request quota (429).
  Implemented call pacing/throttling and batching to operate within the per-minute limit

Results

- 95% data collection automation (eliminated 2-3h daily manual work)
- 0% data gap rate through automation eliminating human errors
- Terraform IaC for the full infra enabled rebuilding the same pipeline in another GCP project within 30 minutes, with 100% change history in Git

Tech Stack

BigQuery, Dataflow, Cloud Functions, Cloud Scheduler, Cloud Storage, Artifact Registry, Terraform, Python, Google Sheets API, Dune API, MongoDB, CloudSQL, Docker

---

Project 3: PLG Stack Monitoring System

Contribution 100% (System design & implementation) 4 months (2024.04 ~ 2024.07)

Problem

No real-time monitoring system for Kubernetes clusters and applications, allowing only reactive incident response with excessive time spent on root cause analysis.

Approach

Built a PLG Stack collecting metrics with Prometheus and logs with Loki, unified visualization through Grafana dashboards.
Real-time monitoring of CPU/memory/network usage and application logs, with automatic Slack alerts on threshold breaches.

Results

- Established proactive incident detection through threshold-based alerts
- Reduced incident response time through real-time monitoring and alerting
- Improved service availability
- Reduced operations team fatigue and improved actual incident detection accuracy through SLI/SLO establishment and Alertmanager alarm noise optimization

Tech Stack

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support.
Designed and built on-premises cloud infrastructure for financial and public sector clients, and led training for client operations teams.

Project 1: Customized PaaS Solution for Clients

Contribution 100% (Infra design & Kubernetes cluster build) 11 months (2022.04 ~ 2023.03)

Problem

Each client had different environment and server spec requirements, making standardized solutions insufficient.

Approach

Designed Kubernetes clusters in HA (High Availability) configuration per client requirements and built virtual machine management environments with OpenStack.
Provided unified block/file/object storage through Ceph and conducted parallel infrastructure operations training for client teams.

Results

- Successfully built and delivered PaaS solutions to 5 clients
- Achieved 99.9% availability with Kubernetes cluster HA configuration
- Ansible-based server provisioning and configuration management automation reducing per-client deployment time and eliminating configuration drift issues

Tech Stack

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

Project 2: Kubernetes Operations Training Program for DP

Contribution 100% (Training design & delivery) 6 months (2022.06 ~ 2022.11)

Problem

DP's operations team lacked Kubernetes and container-based infrastructure experience, anticipating difficulties in self-operation after PaaS solution adoption.

Approach

Designed a step-by-step training curriculum from Kubernetes basics to cluster operations and troubleshooting, with hands-on lab environments for practice-oriented training.
Also conducted parallel training on OpenStack and Ceph storage operations to ensure full PaaS stack operational capability.

Results

- Operated the curriculum with an average of 10 attendees per session and 4 hours per session, establishing DP's autonomous Kubernetes operations system
- Reduced client technical inquiries post-training, achieved self-incident response capability
- Standardized training materials reused for subsequent client trainings

Tech Stack

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

Project 3: Kubernetes Certificate Auto-renewal Process Improvement

Contribution 100% (Solo execution, propagated across all client clusters) 2 weeks (2022.11)

Problem

Kubernetes cluster certificates expired annually, requiring ~5 hours of renewal work per client per year, with service outage risks upon expiration.

Approach

Analyzed the Kubernetes certificate management process and modified kubeadm settings to extend certificate validity from 1 year to 10 years.
Developed automation scripts applicable to existing clusters and deployed to all clients.

Troubleshooting

- Extending validity by editing kubeadm settings only applied to the kubeadm-initialized cluster versions in use at the time; on v1.15.x / v1.16.x a known `kubeadm alpha certs renew` bug left some certificates un-renewed.
  In those cases, rather than relying on the kubeadm command, I found and adopted a public open-source script (update-kube-cert) that backs up `/etc/kubernetes` and regenerates the certificate files directly to 10-year validity, applying it alongside restarts of etcd, apiserver, controller-manager, scheduler, and kubelet so it works regardless of version.

Results

- 10x certificate renewal cycle extension (annual → once per 10 years)
- ~50 hours annual operations savings across 10 clients
- Eliminated outage risks from certificate expiration

Tech Stack

Kubernetes, kubeadm, Bash Script, OpenSSL